

# RoboFusion 1.0 — SCS-RG

Multi-Hazard Smart Campus Safety & Response Grid

System Documentation | Techathon Round 1 | UFTB Robotics Club

Team: The Underdogs | Track B — Simulation (Wokwi ESP32)

## 1. Circuit Diagrams (Test Case 26)

Zone 1 — IoT Lab Equivalent simulated via **Wokwi ESP32**. The three diagrams below show the circuit in each operational state. All sensor wiring, pin assignments, debounce logic and actuator responses are implemented in **sketch/sketch.ino**.

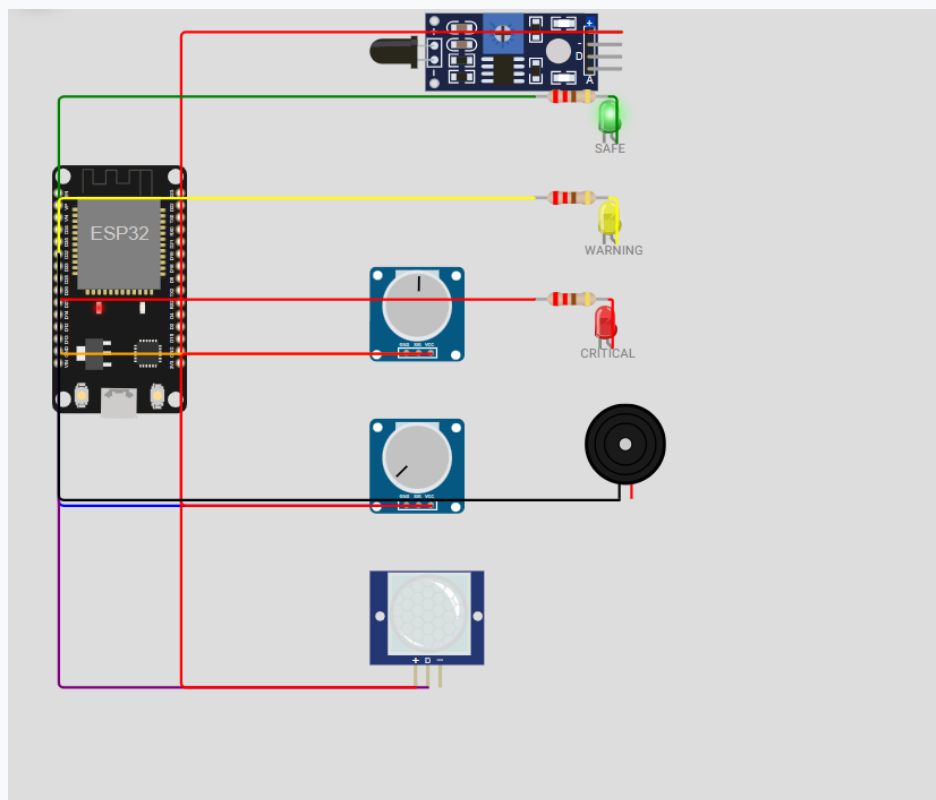


Figure 1 — SAFE State: Green LED ON, Buzzer OFF, all sensors within normal range

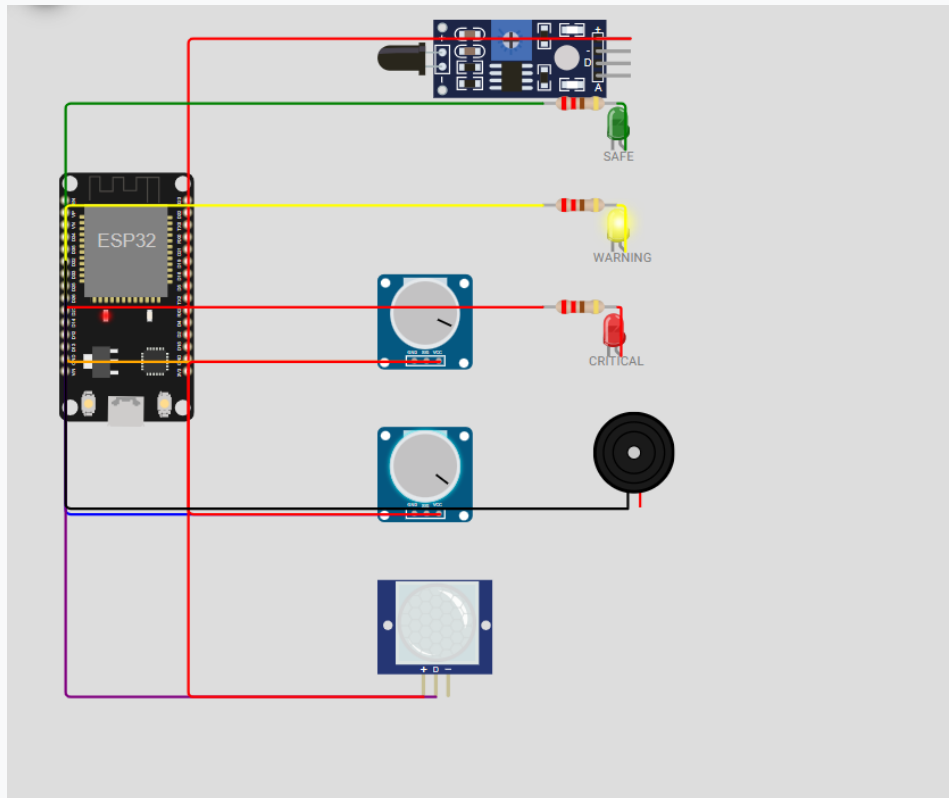


Figure 2 — WARNING State: Yellow LED ON, Buzzer OFF, gas/water elevated above safe threshold

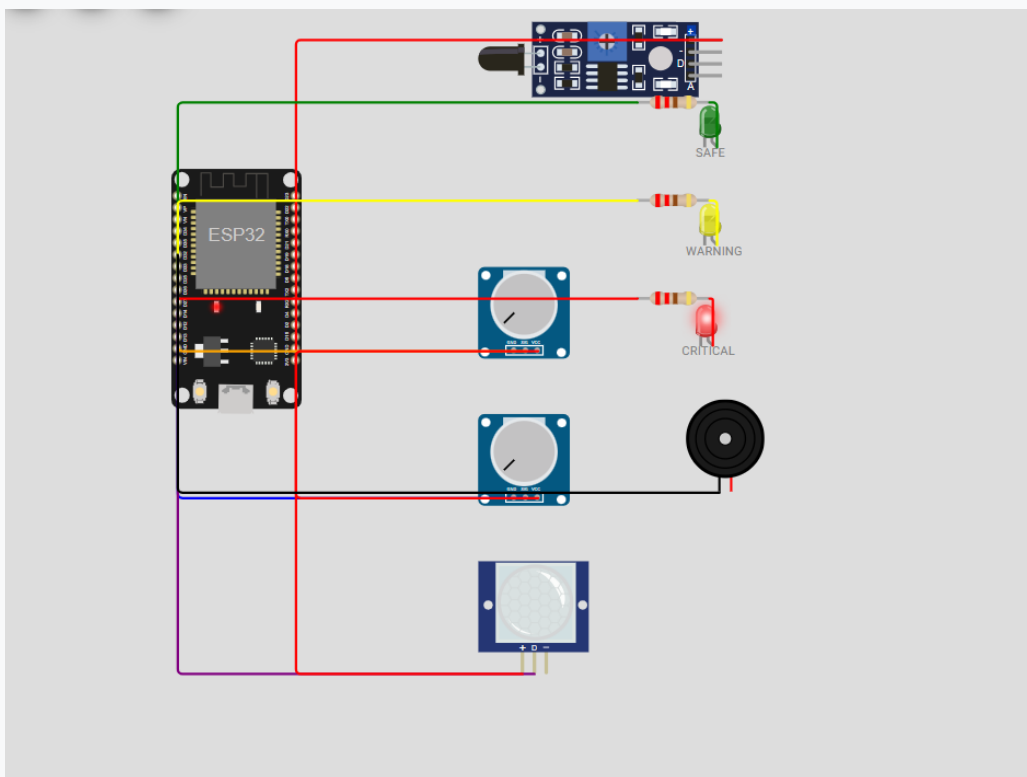


Figure 3 — CRITICAL State: Red LED ON, Buzzer ON, flame confirmed after 5-reading debounce

## Pin Configuration

| Sensor / Actuator    | ESP32 Pin | Type           |
|----------------------|-----------|----------------|
| Flame Sensor         | D13       | Digital Input  |
| Gas Sensor (MQ-2)    | D34       | Analog Input   |
| Water Level Sensor   | D35       | Analog Input   |
| PIR Motion Sensor    | D14       | Digital Input  |
| Green LED (SAFE)     | D25       | Digital Output |
| Yellow LED (WARNING) | D26       | Digital Output |
| Red LED (CRITICAL)   | D27       | Digital Output |
| Buzzer               | D32       | Digital Output |

**Sensor Behaviour**

- Fire sensor sampled every 500 ms — debounce requires **5 consecutive HIGH readings** before confirming flame.
- Gas sensor readings suppressed for **first 30 seconds after boot** (warm-up window) — prevents false trigger on power-on noise.
- Water level normalised to 0.0–1.0; **negative values rejected** as invalid input (Edge Case f).
- PIR retrigger delay prevents log spam on brief exit/re-entry.
- Critical rule:** Zone node sends raw values only — it never computes or reports its own risk score or state.

## 2. Software Architecture (Test Case 27)

```
[Zone Node - Wokwi ESP32] Sensors -> Debounce / Warmup -> Raw values only | | HTTP POST {fire, gas, water, occupancy, zone_id} v [Backend - Express / TypeScript :3000] Risk Fusion Engine -> calculateRiskScore() Priority Ranking -> Zone ordering by risk + occupancy + time Alert Broadcast -> WebSocket + 1.5s polling fallback RBAC -> Role enforcement on every admin endpoint | | Read / Write v [Database - SQLite via sql.js] Zones . Sensors . Readings . Incidents . Acknowledgments . Users . Settings | | REST API / WebSocket v [Frontend Dashboard - React 19 + Vite :5173] Live Zone Map -> Priority Queue -> Incident Timeline Role Views -> ML Risk Panel -> Actuator Controls
```

**Critical architectural rule:** The zone node sends only raw sensor values. Risk score and SAFE/WARNING/CRITICAL state are computed exclusively on the backend and persisted to SQLite. The zone node never decides or reports its own state.

## 3. API Documentation (Test Case 28)

| Endpoint                                | Description                                                          |
|-----------------------------------------|----------------------------------------------------------------------|
| GET /api/v1/zones                       | Returns current state of all zones in a single call                  |
| GET /api/v1/incidents                   | Incident history — supports ?from=&to= date filtering                |
| POST /api/v1/incidents/:id/acknowledge  | Race-condition safe via UNIQUE constraint. Returns 404 if ID missing |
| POST /api/v1/sensor-data                | Zone node raw readings ingestion. Backend computes risk score        |
| POST /api/v1/config/thresholds          | Admin only — returns 403 if caller is not Admin                      |
| GET /api/v1/incidents/critical-last-24h | Indexed CRITICAL query using idx_incidents_status_time               |
| GET /api/v1/health                      | Uptime, zone count, active incidents                                 |

### Example — GET /api/v1/zones response

```
{ "success": true, "data": [{ "id": "z-1", "name": "Chemical Synthesis Lab", "currentState": "CRITICAL", "occupantCount": 6, "currentRiskScore": { "fireScore": 40, "gasScore": 28.5, "totalScore": 68.5, "state": "CRITICAL" }, "mlPrediction": { "probabilityCritical": 0.91, "trendDirection": "rising" } } ] }
```

### Example — POST /api/v1/sensor-data body

```
{ "zoneId": "z-1", "readings": [ { "type": "fire", "rawValue": 1 }, { "type": "gas", "rawValue": 0.72 }, { "type": "water", "rawValue": 0.1 }, { "type": "pir", "rawValue": 1 } ] }
```

## 4. Database Schema (Test Case 29)

| Table           | Key Fields                                                                         |
|-----------------|------------------------------------------------------------------------------------|
| Zones           | id (PK), name, code (UNIQUE), current_state, occupant_count                        |
| Sensors         | id (PK), zone_id (FK->Zones), type, min_value, max_value                           |
| Readings        | id (PK), zone_id (FK ON DELETE RESTRICT), raw_value, normalized_value, received_at |
| Incidents       | id (PK), zone_id (FK ON DELETE RESTRICT), status, start_time, end_time             |
| Acknowledgments | id (PK), incident_id (FK UNIQUE->Incidents), user_id, ack_time                     |
| Users           | id (PK), username (UNIQUE), role CHECK(IN 'Security Staff','Admin')                |
| Settings        | key (PK), value — risk weights + thresholds as JSON                                |

### Key Constraints

- \* Readings.zone\_id -> Zones.id ON DELETE RESTRICT
- \* Incidents.zone\_id -> Zones.id ON DELETE RESTRICT
- \* Acknowledgments.incident\_id UNIQUE — race-condition safety, only one ack wins
- \* INDEX idx\_incidents\_status\_time ON Incidents(status, start\_time) — fast 24h CRITICAL query

## 5. Risk Fusion Formula (Test Case 30)

```
risk_score = (0.4 x fire) + (0.3 x gas_norm) + (0.2 x water_norm) + (0.1 x occupancy) SAFE:
score < 30 WARNING: 30 <= score < 60 CRITICAL: score >= 60 (Matches DEFAULT_THRESHOLDS in
riskCalculator.ts: safeUpperLimit=30, warningUpperLimit=60)
```

| Sensor    | Weight | Justification                                                                   |
|-----------|--------|---------------------------------------------------------------------------------|
| Fire      | 0.4    | Highest — escalates fastest, most immediate damage in electronics/chemicals lab |
| Gas       | 0.3    | Toxic/flammable gas is immediate life threat, can precede explosion             |
| Water     | 0.2    | Electrical short-circuit and equipment damage; slower escalation                |
| Occupancy | 0.1    | Low per-zone; but weighs heavily in cross-zone priority ranking                 |

## 6. Backup & Recovery Strategy (Test Case 20)

**Backup:** SQLite exported via sql.js .export() buffer every 30 minutes, written to robofusion\_backup\_YYYYMMDD.db.

**Recovery:** Stop backend, copy backup to robofusion.db, restart. initDB() reconstructs all zone/incident/user state. Only data since last backup is lost.

## 7. Data Retention & Access Policy (Test Case 21)

Raw Readings older than **90 days** are summarised and dropped.

Incidents (audit trail) kept indefinitely.

**Admin:** can query raw historical readings. **Security Staff:** sees status and incident timeline only.

## 8. Scalability for 30+ Zones (Test Case 11b)

---

| Change                      | Reason                                                |
|-----------------------------|-------------------------------------------------------|
| Migrate to PostgreSQL       | Real concurrent write throughput + connection pooling |
| Add Redis                   | Cross-instance WebSocket pub/sub for multiple workers |
| Load balancer + PM2 cluster | Multiple backend workers share traffic                |
| Separate ML microservice    | Keeps prediction from blocking ingestion path         |

## 9. ML Risk Prediction (Bonus 3)

---

Trained on synthetic sensor history generated from the risk fusion formula.

Model: logistic regression over a rolling window of normalised sensor readings.

Outputs: probabilityCritical (0.0-1.0) and trendDirection (rising/falling/stable).

Shown as a **separate Predicted Risk panel** — never mixed with the live risk score.

**Safety statement:** Predicted value can NEVER trigger relay, buzzer, or actuator. All actuation is gated exclusively on the live server-side risk score.